

Actividad [Individual] API REST MVC

Resultados de Aprendizaje y Criterios de Evaluación aplicados

RA5	Desarrolla aplicaciones web identificando y aplicando mecanismos para separar el código de presentación de la lógica de negocio.
a	Se han identificado las ventajas de separar la lógica de negocio de los aspectos de presentación de la aplicación.
b	Se han analizado y utilizado mecanismos y frameworks que permiten realizar esta separación y sus características principales.
c	Se han utilizado objetos y controles en el servidor para generar el aspecto visual de la aplicación web en el cliente.
d	Se han utilizado formularios generados de forma dinámica para responder a los eventos de la aplicación web.
e	Se han identificado y aplicado los parámetros relativos a la configuración de la aplicación web.
f	Se han escrito aplicaciones web con mantenimiento de estado y separación de la lógica de negocio.
g	Se han aplicado los principios y patrones de diseño de la programación orientada a objetos.
h	Se ha probado y documentado el código.

En el enunciado se expone la relación de cada punto con su criterio de evaluación (Ejemplo: CE b).

Enunciado

Aplica los siguientes cambios sobre la actividad anterior “**Act - Node JS Express EJS - Star Wars**” (o, en caso de no disponer de ella, crea un nuevo BackEnd):

- Utiliza el patrón MVC (Modelo – Vista – Controlador): **(CE g)**
 - Directorio “**Models**” que almacenará el modelo de datos, en este caso, únicamente el archivo “**starwars.json**”. **(CE f)**
 - Directorio “**Controllers**”, que almacenará el controlador “**SWController**” con las funciones que serán ejecutadas por el servicio, a partir de las rutas, y devolverán una respuesta (response) a partir de una petición (request). **(CE f)**
 - Directorio “**Services**”, que almacenará el servicio “**SWService**” y que implementará las funciones con la lógica de negocio necesaria por parte del controller. Deberá devolver la respuesta adecuada al este. **(CE f)**
 - Directorio “**Routes**”, que almacenará las rutas (“**SWRoutes**”) de acceso al API REST y llamará al controlador anterior. **(CE f)**
 - Directorio “**Views**” con las siguientes vistas necesarias (muchas ya existentes de la anterior actividad). Modifícalas y adapítalas si lo consideras necesario (añade mejoras visuales CSS, accesibilidad, navegación satisfactoria, experiencia de usuario...): **(CE abcd)**
 - Listar personajes
 - Ver detalles de un personaje (con botón de editar y eliminar)
 - Editar un personaje
 - Crear un nuevo personaje
- Crea un archivo **.ENV** para almacenar todas aquellas variables de entornos que consideres (por ejemplo, el puerto de la aplicación y cualquier otra que consideréis). Úsalas en tu API REST. **(CE e)**
- Documenta, mediante comentarios en código, la máxima cantidad de información posible dentro del API REST. **(CE h)**
- Realiza todas las modificaciones oportunas en el servidor de Node JS (**index.js**, import/export...) de tal manera que el BackEnd funcione perfectamente. **Las operativas CRUD deben funcionar al 100%**. **(CE h)**

- **Entrega:**
 - Utiliza **GIT** para trabajar en grupos. Sube tu repositorio a un GIT remoto (Bitbucket, Github...) y comparte la URL y Password o Token para poder hacer un "git clone". Para ello, sube un archivo TXT o PDF con dicha información a Aules.
 - Además, comprime tu carpeta en formato ZIP y súbela a Aules, por si el repositorio GIT remoto fallase, con el siguiente formato: **Act_SW_Rest_Nombre_Apellidos.zip** (solo un miembro del grupo).
 - **Se penalizará el incumplimiento de estas normas de entrega.**
- **Rúbricas → Ver calificación en Aules**

***Autoevaluación y Coevaluación**

Vinculado a la tarea, existirá un cuestionario de autoevaluación y coevaluación del grupo. De esta forma, cada integrante podrá autoevaluarse explicando qué ha hecho y podrá coevaluar a sus compañeros.

Este cuestionario es totalmente confidencial y servirá para equilibrar el peso de la calificación final entre los miembros del equipo.